

MODIS EVI Data Analysis

This vignette demonstrates the real-data workflow for the cleaned MODIS EVI reviewer subset distributed in the BSGl GitHub repository. The example follows the paper preprocessing: continuous predictors are scaled using the training set, the land-cover variable is represented by main-effect dummy variables, and EVI is modeled on the log scale.

The full raw MODIS data are not included. The prepared reviewer subset is stored in the GitHub source repository as `data/data_cleaned_small_expanded.RData`. It is intentionally excluded from the built package because of its size; the code below follows `run_real_demo.R` by using installed `extdata` when available and otherwise downloading only this prepared file. The demo refits models locally rather than reading manuscript tables directly. The retained CSV files under `results/` provide the manuscript-reference values reported in the paper.

Required Packages

```
## =====  
## load packages  
## =====  
library(BSGL)  
library(ggplot2)  
library(cowplot)  
library(viridis)
```

MODIS Data

The real-data subset contains EVI, reflectance variables, productivity and energy variables, viewing geometry, land-cover type, and scaled spatial coordinates.

```
## =====  
## load the data  
## =====  
data_file <- system.file(  
  "extdata", "real_demo", "data_cleaned_small_expanded.RData",  
  package = "BSGL"  
)  
  
if (!nzchar(data_file) || !file.exists(data_file)) {  
  demo_dir <- file.path(tempdir(), "BSGL_real_demo")  
  dir.create(demo_dir, recursive = TRUE, showWarnings = FALSE)  
  data_file <- file.path(demo_dir, "data_cleaned_small_expanded.RData")  
  if (!file.exists(data_file)) {  
    utils::download.file(  
      paste0(  
        "https://raw.githubusercontent.com/Qishi7/BSGL/",  
        "main/data/data_cleaned_small_expanded.RData"  
      ),  
      data_file,  
      mode = "wb"  
    )  
  }  
}
```

```

    )
  }
}

if (!file.exists(data_file)) {
  stop("Prepared MODIS reviewer data could not be located or downloaded.")
}

load(data_file)
real_data <- data_cleaned_small

dim(real_data)
names(real_data)

```

Preprocessing

The helper code below constructs the 10-variable design matrix used in the real-data analysis. The first nine variables are continuous main effects. The land-cover variable `LC_Type4` is kept as the tenth main effect through dummy coding relative to the first observed training level.

```

## =====
## preprocessing helpers
## =====

scale01_train_test <- function(train_x, test_x) {
  train_scaled <- train_x
  test_scaled <- test_x
  for (j in seq_len(ncol(train_x))) {
    lo <- min(train_x[, j], na.rm = TRUE)
    hi <- max(train_x[, j], na.rm = TRUE)
    denom <- hi - lo
    if (!is.finite(denom) || denom == 0) {
      train_scaled[, j] <- 0
      test_scaled[, j] <- 0
    } else {
      train_scaled[, j] <- (train_x[, j] - lo) / denom
      test_scaled[, j] <- (test_x[, j] - lo) / denom
    }
  }
  colnames(train_scaled) <- colnames(train_x)
  colnames(test_scaled) <- colnames(test_x)
  list(train = train_scaled, test = test_scaled)
}

make_lc_dummy_design <- function(train_lc, test_lc) {
  train_levels <- sort(unique(train_lc))
  ref_level <- train_levels[1]
  dummy_levels <- train_levels[train_levels != ref_level]
  train_dummy <- sapply(dummy_levels, function(level) as.numeric(train_lc == level))
  test_dummy <- sapply(dummy_levels, function(level) as.numeric(test_lc == level))
  if (length(dummy_levels) == 1) {
    train_dummy <- matrix(train_dummy, ncol = 1)
    test_dummy <- matrix(test_dummy, ncol = 1)
  }
  colnames(train_dummy) <- paste0("LC_Type4_", dummy_levels)
}

```

```

colnames(test_dummy) <- paste0("LC_Type4_", dummy_levels)
list(train = train_dummy, test = test_dummy,
      reference = ref_level, levels = train_levels)
}

## =====
## construct training and test sets
## =====
sample_size <- 1000
set.seed(20260531)
idx <- if (nrow(real_data) > sample_size) {
  sample(nrow(real_data), sample_size)
} else {
  seq_len(nrow(real_data))
}
sample_data <- real_data[idx, ]

set.seed(456)
test_indices <- sample(seq_len(nrow(sample_data)),
                      size = floor(0.2 * nrow(sample_data)))
train_indices <- setdiff(seq_len(nrow(sample_data)), test_indices)

# Keep the spatial boundary in the training set, matching run_real_demo.R.
boundary_indices <- unique(c(
  which(sample_data$scaled_x == min(sample_data$scaled_x)),
  which(sample_data$scaled_x == max(sample_data$scaled_x)),
  which(sample_data$scaled_y == min(sample_data$scaled_y)),
  which(sample_data$scaled_y == max(sample_data$scaled_y))
))
test_indices <- setdiff(test_indices, boundary_indices)
train_indices <- sort(unique(c(train_indices, boundary_indices)))

continuous_vars <- c(
  "red_reflectance", "NIR_reflectance", "blue_reflectance",
  "MIR_reflectance", "GPP", "LE", "view_zenith_angle",
  "sun_zenith_angle", "relative_azimuth_angle"
)

train_y <- log(sample_data$EVI[train_indices] + 1)
test_y <- log(sample_data$EVI[test_indices] + 1)

scaled_cont <- scale01_train_test(
  as.matrix(sample_data[train_indices, continuous_vars]),
  as.matrix(sample_data[test_indices, continuous_vars])
)
lc <- make_lc_dummy_design(
  sample_data$LC_Type4[train_indices],
  sample_data$LC_Type4[test_indices]
)

train_X <- cbind(scaled_cont$train, lc$train)
test_X <- cbind(scaled_cont$test, lc$test)
train_coords <- as.matrix(sample_data[train_indices, c("scaled_x", "scaled_y")])
test_coords <- as.matrix(sample_data[test_indices, c("scaled_x", "scaled_y")])

```

```
dim(train_X)
colnames(train_X)
```

Model Fits

The runnable real-data example fits BSGL, GSVC, and GGP-GAM on the same training set. BSGL and GSVC also provide posterior coefficient-surface credible intervals for SCP summaries. MGWR can be attempted as an optional local comparison fit; the package installation and default real-data demo do not run MGWR because `GWmodel::gwr.multiscale()` can be slow on real-data subsets. Set `REAL_DEMO_RUN_MGWR=true` when running `run_real_demo.R` to attempt the optional local MGWR fit. The optional MGWR run uses warm-start bandwidth initialization and skips automatically when the training set exceeds `REAL_DEMO_MGWR_MAX_TRAIN` (default: 1000). MGWR is included in the retained manuscript-reference real-data tables.

```
## =====
## fit BSGL, GSVC, and GGP-GAM
## =====
set.seed(20260531)

bsgl_fit <- fit_bsgl_multichain(
  y = train_y,
  X = train_X,
  coords = train_coords,
  L = 25,
  niter = 800,
  nburn = 200,
  a_lam = 30,
  b_lam = 0.1,
  nchains = 2,
  parallel = FALSE,
  verbose = TRUE
)

gs_fit <- fit_gs(
  y = train_y,
  X = train_X,
  coords = train_coords,
  L = 25,
  niter = 800,
  nburn = 200,
  verbose = FALSE
)

gam_fit <- fit_gam(train_X, train_y, train_coords)
```

MGWR can be run locally when desired:

```
## =====
## optional MGWR local comparison
## =====
mgwr_var_n <- ncol(train_X) + 1L
mgwr_fit <- fit_mgwr(
  train_X,
  train_y,
  train_coords,
```

```

    bws0 = rep(length(train_y) - 1L, mgwr_var_n),
    bw.seled = rep(TRUE, mgwr_var_n),
    hatmatrix = FALSE
)

```

Prediction

```

## =====
## four-method prediction comparison
## =====
bsgl_pred <- pred_bsgl(bsgl_fit, test_X, test_coords)
gs_pred <- pred_gs(gs_fit, test_X, test_coords)
gam_pred <- pred_gam(gam_fit, test_X, test_coords)

prediction <- data.frame(
  Method = c("BSGL", "GSVC", "GGP-GAM"),
  MSPE = c(
    mean((test_y - bsgl_pred$mean)^2),
    mean((test_y - gs_pred$mean)^2),
    mean((test_y - gam_pred)^2)
  ),
  MAE = c(
    mean(abs(test_y - bsgl_pred$mean)),
    mean(abs(test_y - gs_pred$mean)),
    mean(abs(test_y - gam_pred))
  )
)

prediction

```

SCP Summary

```

## =====
## real-data SCP
## =====
grid_data <- list(grid_points = train_coords)
bsgl_scp <- calc_scp_bsgl(bsgl_fit, grid_data, ci_level = 0.95)
gs_scp <- calc_scp_gs(gs_fit, grid_data, ci_level = 0.95)

scp <- data.frame(
  variable = colnames(train_X),
  BSGL = as.numeric(bsgl_scp),
  GSVC = as.numeric(gs_scp)
)

scp

```

The prediction and SCP objects above are local demo results. Small numerical differences across local runs may occur due to runtime settings, stochastic fitting components, or seed/split differences.

Standalone Reviewer Workflow

For the complete reviewer-facing workflow, including independent method-status reporting and the common prediction table, run:

```
source(  
  "https://raw.githubusercontent.com/Qishi7/BSGL/main/run_real_demo.R"  
)
```